

NOMBRES: ALEJANDRO JOSE
APELLIDOS: GOMEZ HERNANDEZ
CARNE: 20347
FECHA: 25/08/2024

Ejercicio 1 Con los datos de la clase de lunes, calcule PPMI, pero aplicando Laplace Smoothing.

```
In [ ]: import pandas as pd

df = pd.read_excel('pmi_ejercicio.xlsx')
print(df.head())

   Unnamed: 0  Unnamed: 1  Unnamed: 2  Unnamed: 3  Unnamed: 4  Unnamed: 5
0         NaN      Computer      data      result      pie      sugar
1      Cherry          2          8          9      44260        25
2  Strawberry          0          0          1          5        19
3    Digital      1670      1683          85          5          4
4  Information      3325      3982         378         512         13

In [ ]: cooccurrence_matrix = df.iloc[1:, 1:].apply(pd.to_numeric, errors='coerce').values

In [ ]: import numpy as np

In [ ]: total_occurrences = np.sum(cooccurrence_matrix)

In [ ]: print(total_occurrences)
55986

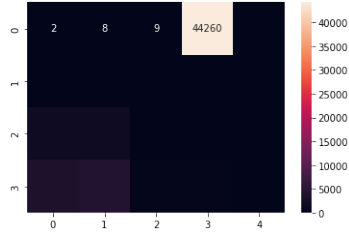
In [ ]: row_sums = np.sum(cooccurrence_matrix, axis=1)
col_sums = np.sum(cooccurrence_matrix, axis=0)

In [ ]: print("Suma de filas: ", row_sums)
print("Suma de columnas: ", col_sums)
Suma de filas: [44304  25 3447 8210]
Suma de columnas: [ 4997 5673 473 44782  61]

In [ ]: pmi_matrix = np.zeros_like(cooccurrence_matrix, dtype=np.float64)

In [ ]: import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(cooccurrence_matrix, annot=True, fmt='d')
```

Out []: <Axes: >



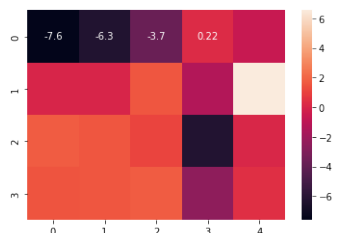
```
In [ ]: for i in range(cooccurrence_matrix.shape[0]):
    for j in range(cooccurrence_matrix.shape[1]):
        joint_prob = cooccurrence_matrix[i, j] / total_occurrences
        if joint_prob > 0:
            p_i = row_sums[i] / total_occurrences
            p_j = col_sums[j] / total_occurrences
            pmi_matrix[i, j] = np.log(joint_prob / (p_i * p_j))

In [ ]: print(pmi_matrix)

sns.heatmap(pmi_matrix, annot=True)

[[-7.58941914 -6.33000512 -3.72784412  0.22230175 -0.65797135]
 [ 0.          0.          1.55488573 -1.38614253  6.54754623]
 [ 1.69158327  1.57245721  1.07115324 -6.31252627  0.06301787]
 [ 1.51238068  1.56581486  1.69554754 -2.55148819  0.37382423]]

Out [ ]: <Axes: >
```



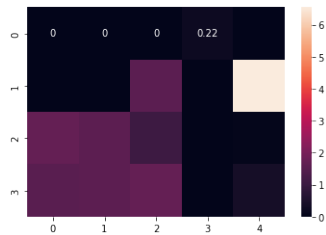
```
In [ ]: ppmi_matrix = np.maximum(pmi_matrix, 0)

In [ ]: print(ppmi_matrix)

[[0.          0.          0.          0.22230175  0.          ]
 [0.          0.          1.55488573  0.          6.54754623]
 [1.69158327  1.57245721  1.07115324  0.          0.06301787]
 [1.51238068  1.56581486  1.69554754  0.          0.37382423]]

In [ ]: sns.heatmap(ppmi_matrix, annot=True)

Out [ ]: <Axes: >
```



```
In [ ]: ppmi_df = pd.DataFrame(ppmi_matrix, index=df.iloc[1:, 0].values, columns=df.iloc[0, 1:].values)
```

```
In [ ]: print(ppmi_df)
```

```
Computer      data      result      pie      sugar
Cherry      0.000000  0.000000  0.000000  0.222302  0.000000
Strawberry   0.000000  0.000000  1.554886  0.000000  6.547546
Digital      1.691583  1.572457  1.071153  0.000000  0.063018
Information   1.512381  1.565815  1.695548  0.000000  0.373824
```

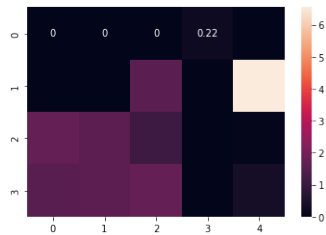
```
In [ ]: import scipy as sp
```

```
PPMI_sparse = sp.sparse.csr_matrix(ppmi_matrix)
print(PPMI_sparse)
```

```
(0, 3)      0.22230175470509048
(1, 2)      1.554885725101943
(1, 4)      6.547546228587005
(2, 0)      1.6915832660118073
(2, 1)      1.5724572088094504
(2, 2)      1.071153239943595
(2, 4)      0.063017868891791
(3, 0)      1.5123806777613524
(3, 1)      1.5658148633131774
(3, 2)      1.6955475431432576
(3, 4)      0.37382422930382847
```

```
In [ ]: sns.heatmap(PPMI_sparse.toarray(), annot=True)
```

```
Out [ ]: <Axes: >
```



Ejercicio 2

POC para crear información de entreno

```
In [ ]: #Librerías que necesitarán
import io
import re
import string
import tqdm

import numpy as np

import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras import losses
from tensorflow.keras import preprocessing
from tensorflow.keras import utils
%load_ext tensorboard
SEED = 42
AUTOTUNE = tf.data.AUTOTUNE
```

```
In [ ]: import nltk
nltk.download('stopwords')
sentence = "The wind Crosses the brown land unheard" #De mis poemas fav. The waste land de T.S Elliot
#tokens = #tokenizar
tokens = sentence.split()
tokens = [word.lower() for word in tokens]
tokens = [re.sub(r'[\w\s]', '', word) for word in tokens]
tokens = [word for word in tokens if word.isalpha()]
tokens = [word for word in tokens if word not in string.punctuation]

print(tokens)

vocab, index = {}, 1

vocab["algo importante para que no explote el programa"] = 0

#Llene el arreglo para vocabulario
for word in tokens:
    if word not in vocab:
        vocab[word] = index
        index += 1

vocab_size = len(vocab)
print("Tamaño vocab:", vocab_size)
print("Vocab:", vocab)

vocab_size = len(vocab)
example_sequence = [vocab[word] for word in tokens]
window_size = 2
positive_skip_grms, _ = tf.keras.preprocessing.sequence.skipgrams(
    example_sequence,
    vocabulary_size=vocab_size,
    window_size=window_size,
    negative_samples=0
)
```

```

print(len(positive_skip_grams))
print(positive_skip_grams)

['the', 'wind', 'crosses', 'the', 'brown', 'land', 'unheard']
Tamaño vocab: 7
Vocab: {'algo importante para que no explote el programa': 0, 'the': 1, 'wind': 2, 'crosses': 3, 'brown': 4, 'land': 5, 'unheard': 6}
22
[[4, 3], [5, 1], [1, 4], [2, 1], [1, 2], [3, 1], [4, 1], [1, 5], [2, 1], [1, 3], [3, 1], [5, 6], [1, 2], [3, 2], [6, 4], [3, 4], [6, 5], [5, 4], [4, 6], [2, 3], [4, 5], [1, 3]]
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

```

In [ ]: #Resultados
inverse_vocab = {index: token for token, index in vocab.items()}
for target, context in positive_skip_grams[:5]:
    print(f"({target}), ({context}): ({inverse_vocab[target]}, {inverse_vocab[context]})")

(4, 3): (brown, crosses)
(5, 1): (land, the)
(1, 4): (the, brown)
(2, 1): (wind, the)
(1, 2): (the, wind)

```

```

In [ ]: target_word, context_word = positive_skip_grams[0]
# Escoge un numero para muestras negativos (que no pertenece al contexto)
num_ns = 4

context_class = tf.reshape(tf.constant(context_word, dtype="int64"), (1, 1))
negative_sampling_candidates, _, _ = tf.random.log_uniform_candidate_sampler(
    true_classes=context_class,
    num_true=1,
    num_sampled=num_ns,
    unique=True,
    range_max=vocab_size,
    seed=SEED,
    name="negative_sampling"
)

```

```

In [ ]: print(negative_sampling_candidates)
print([inverse_vocab[index.numpy()] for index in negative_sampling_candidates])

tf.Tensor([2 1 4 3], shape=(4,), dtype=int64)
['wind', 'the', 'brown', 'crosses']

```

```

In [ ]: sampling_table = tf.keras.preprocessing.sequence.make_sampling_table(vocab_size)
print(sampling_table)

[0.00315225 0.00315225 0.00547597 0.00741556 0.00912817 0.01068435
 0.01212381]

```

```

In [ ]: negative_sampling_candidates = tf.expand_dims(negative_sampling_candidates, 1) #se agrega una dimension para poder concatenar
context = tf.concat([context_class, negative_sampling_candidates], 0)
label = tf.constant([1] + [0]*num_ns, dtype="int64")
target = tf.squeeze(target_word)
context = tf.squeeze(context)
label = tf.squeeze(label)

#Listo! asi se prepara la info para entrenar

```

```

In [ ]: #Informacion a utilizar:
path_to_file = tf.keras.utils.get_file('shakespeare.txt', 'https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt')
text_ds = tf.data.TextLineDataset(path_to_file).filter(lambda x: tf.cast(tf.strings.length(x), bool))

Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt
1115394/1115394 ----- 0s 0us/step

```

```

In [ ]: #Paso 1: cree una funcion que estandarice el texto como ya hemos hecho e incrustela en una capa de Tensorflow (TextVectorization)

def custom_standardization(input_text):
    lowercase = tf.strings.lower(input_text)
    stripped_html = tf.strings.regex_replace(lowercase, '[%s]' % re.escape(string.punctuation), '')
    no_digits = tf.strings.regex_replace(stripped_html, '\d+', '')
    return no_digits

```

```

In [ ]: vocab_size = 10000
sequence_length = 10

```

```

In [ ]: vectorize_layer = layers.TextVectorization(
    max_tokens=vocab_size,
    output_mode='int',
    output_sequence_length=sequence_length,
    standardize=custom_standardization
)

```

```

In [ ]: vectorize_layer.adapt(text_ds.batch(1024))

```

```

In [ ]: #Paso 2: Usando la POC de creacion de datos de entreno, defina una funcion para este proceso

```

```

def generate_training_data(sequences, window_size, num_ns, vocab_size, vectorize_layer):
    targets = []
    contexts = []
    labels = []

    for seq in sequences:
        tokenized_sequence = vectorize_layer(tf.expand_dims(seq, 0))

        positive_skip_grams, _ = tf.keras.preprocessing.sequence.skipgrams(
            tokenized_sequence.numpy().flatten(),
            vocabulary_size=vocab_size,
            window_size=window_size,
            negative_samples=0
        )

        for target_word, context_word in positive_skip_grams:
            context_class = tf.reshape(tf.constant([context_word], dtype="int64"), (1, 1))

            negative_sampling_candidates, _, _ = tf.random.log_uniform_candidate_sampler(
                true_classes=context_class,
                num_true=1,
                num_sampled=num_ns,
                unique=True,
                range_max=vocab_size,
                seed=SEED,
                name="negative_sampling"
            )

            negative_sampling_candidates = tf.expand_dims(negative_sampling_candidates, 1)
            context = tf.concat([context_class, negative_sampling_candidates], 0)

            label = tf.constant([1] + [0]*num_ns, dtype="int64")

```

```

        targets.append(target_word)
        contexts.append(tf.squeeze(context))
        labels.append(tf.squeeze(label))

    return targets, contexts, labels

window_size = 2
num_ns = 4

targets, contexts, labels = generate_training_data(text_ds, window_size, num_ns, vocab_size, vectorize_layer)

```

```

In [ ]: SEED = 42
AUTOTUNE = tf.data.AUTOTUNE
# Vectorize the data in text_ds.
text_vector_ds = text_ds.batch(1024).prefetch(AUTOTUNE).map(vectorize_layer).unbatch()
sequences = list(text_vector_ds.as_numpy_iterator())

```

```

In [ ]: #El siguiente código es el modelo Word2Vec usando lo que ya han hecho. Sin embargo, deben agregar la métrica de similitud que vimos en clase, a la cual deben llamar dots
class Word2Vec(tf.keras.Model):
    def __init__(self, vocab_size, embedding_dim):
        super(Word2Vec, self).__init__()
        self.target_embedding = layers.Embedding(vocab_size,
                                                  embedding_dim,
                                                  name="w2v_embedding")
        self.context_embedding = layers.Embedding(vocab_size,
                                                  embedding_dim)

    def call(self, pair):
        target, context = pair
        # target: (batch, dummy?) # The dummy axis doesn't exist in TF2.7+
        # context: (batch, context)
        if len(target.shape) == 2:
            target = tf.squeeze(target, axis=1)
        # target: (batch,)
        word_emb = self.target_embedding(target)
        # word_emb: (batch, embed)
        context_emb = self.context_embedding(context)
        dots = tf.matmul(context_emb, word_emb[:, tf.newaxis], transpose_b=True)
        dots = tf.squeeze(dots, axis=-1)

        # dots: (batch, context)
        return dots

```

```

In [ ]: #Entrenar modelo y definir CategoricalCrossEntropy como función de pérdida. Mostrar el print de accuracy y loss. Si deciden presentar sus resultados en tensorboard, adjuntar captura

```

```

In [ ]: BATCH_SIZE = 1024
BUFFER_SIZE = 10000
embedding_dim = 128

targets = tf.constant(targets, dtype=tf.int64)
contexts = tf.constant([c.numpy() for c in contexts], dtype=tf.int64)
labels = tf.constant([l.numpy() for l in labels], dtype=tf.int64)

dataset = tf.data.Dataset.from_tensor_slices(((targets, contexts), labels))
dataset = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE, drop_remainder=True)

word2vec = Word2Vec(vocab_size=vocab_size, embedding_dim=embedding_dim)

word2vec.compile(optimizer='adam',
                 loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
                 metrics=['accuracy'])

log_dir = "logs/word2vec"
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)

history = word2vec.fit(dataset, epochs=20, callbacks=[tensorboard_callback])

for i, (loss, accuracy) in enumerate(zip(history.history['loss'], history.history['accuracy'])):
    print(f"Epoch {i+1}, Loss: {loss:.4f}, Accuracy: {accuracy:.4f}")

```

```

Epoch 1/20 606/606 29s 43ms/step - accuracy: 0.3195 - loss: 1.5282
Epoch 2/20 606/606 40s 42ms/step - accuracy: 0.4300 - loss: 1.3605
Epoch 3/20 606/606 42s 44ms/step - accuracy: 0.4788 - loss: 1.2791
Epoch 4/20 606/606 25s 42ms/step - accuracy: 0.5127 - loss: 1.2153
Epoch 5/20 606/606 41s 42ms/step - accuracy: 0.5407 - loss: 1.1600
Epoch 6/20 606/606 44s 47ms/step - accuracy: 0.5646 - loss: 1.1102
Epoch 7/20 606/606 26s 42ms/step - accuracy: 0.5852 - loss: 1.0649
Epoch 8/20 606/606 41s 42ms/step - accuracy: 0.6033 - loss: 1.0242
Epoch 9/20 606/606 41s 43ms/step - accuracy: 0.6184 - loss: 0.9878
Epoch 10/20 606/606 25s 41ms/step - accuracy: 0.6318 - loss: 0.9553
Epoch 11/20 606/606 41s 42ms/step - accuracy: 0.6427 - loss: 0.9260
Epoch 12/20 606/606 41s 41ms/step - accuracy: 0.6524 - loss: 0.9001
Epoch 13/20 606/606 25s 41ms/step - accuracy: 0.6603 - loss: 0.8768
Epoch 14/20 606/606 43s 44ms/step - accuracy: 0.6679 - loss: 0.8561
Epoch 15/20 606/606 40s 42ms/step - accuracy: 0.6737 - loss: 0.8376
Epoch 16/20 606/606 25s 40ms/step - accuracy: 0.6793 - loss: 0.8210
Epoch 17/20 606/606 42s 42ms/step - accuracy: 0.6838 - loss: 0.8063
Epoch 18/20 606/606 40s 41ms/step - accuracy: 0.6877 - loss: 0.7929
Epoch 19/20 606/606 42s 43ms/step - accuracy: 0.6910 - loss: 0.7810
Epoch 20/20 606/606 25s 41ms/step - accuracy: 0.6947 - loss: 0.7702
Epoch 1, Loss: 1.4762, Accuracy: 0.3523
Epoch 2, Loss: 1.3425, Accuracy: 0.4409
Epoch 3, Loss: 1.2677, Accuracy: 0.4846
Epoch 4, Loss: 1.2070, Accuracy: 0.5163
Epoch 5, Loss: 1.1532, Accuracy: 0.5431
Epoch 6, Loss: 1.1047, Accuracy: 0.5662
Epoch 7, Loss: 1.0609, Accuracy: 0.5862
Epoch 8, Loss: 1.0213, Accuracy: 0.6035
Epoch 9, Loss: 0.9858, Accuracy: 0.6180
Epoch 10, Loss: 0.9540, Accuracy: 0.6308
Epoch 11, Loss: 0.9257, Accuracy: 0.6414
Epoch 12, Loss: 0.9004, Accuracy: 0.6510
Epoch 13, Loss: 0.8778, Accuracy: 0.6587
Epoch 14, Loss: 0.8577, Accuracy: 0.6659
Epoch 15, Loss: 0.8396, Accuracy: 0.6718
Epoch 16, Loss: 0.8235, Accuracy: 0.6769
Epoch 17, Loss: 0.8089, Accuracy: 0.6817
Epoch 18, Loss: 0.7959, Accuracy: 0.6854
Epoch 19, Loss: 0.7841, Accuracy: 0.6891
Epoch 20, Loss: 0.7734, Accuracy: 0.6923

```

In []: `%tensorboard --logdir logs/word2vec`

